FPT SCHOOL OF BUSINESS & TECHNOLOGY

AI - MACHINE LEARNING

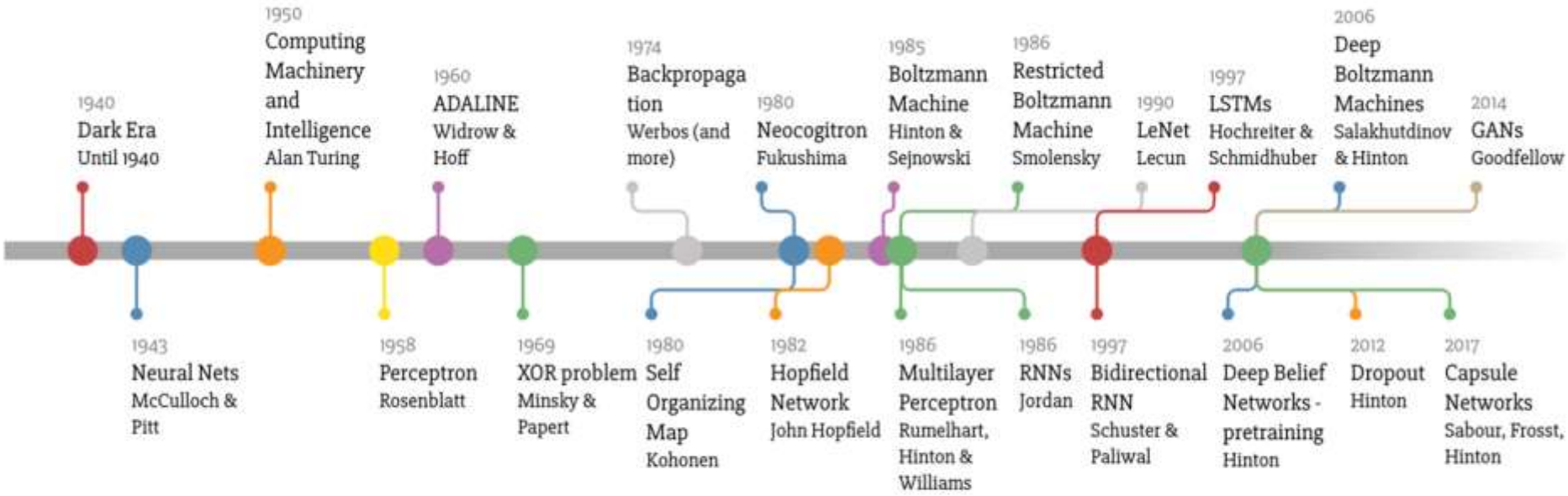
PhD. Truc Thi Kim, Nguyen

093 293 9981 info@fsb.edu.vn https://fsb.edu.vn

DEEP LEARNING

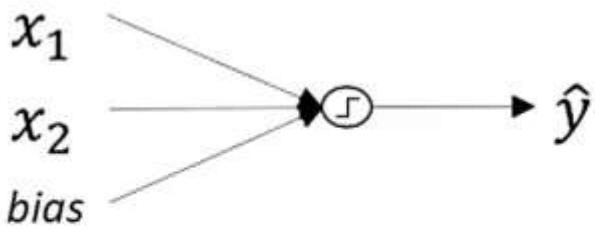
FSB

DL History

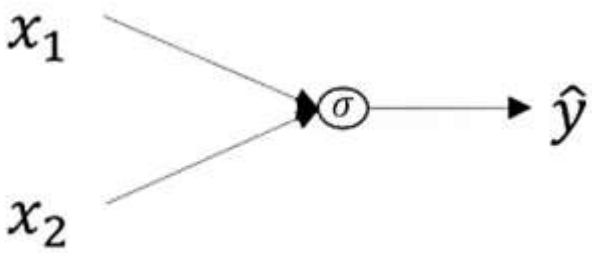


Made by Favio Vázquez

Deep L-layer Neural Network



perceptron



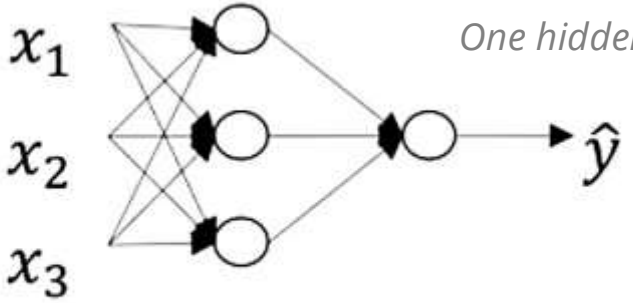
logistic regression

- Perceptron is the simplest version of a Neural Network. The perceptron is a one layer Neural Network with the step activation function.
- Logistic Regression as a single unit that uses sigmoid activation function.
- Both of these simple Neural Networks we also call shallow neural networks and they are only reasonable to be applied when classifying linearly separable classes.

FSB

Deep L-layer Neural Network

- Slightly more complex neural network is a two layer neural network (it is a neural network with one hidden layer). This shallow neural network can classify two datasets that are not linearly separable, but it is not good at classifying more complex datasets.



One hidden layer Neural Network

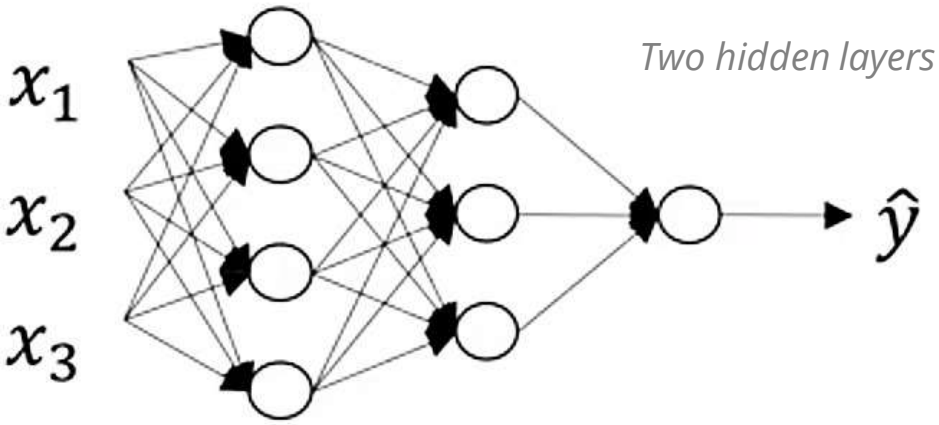
1 hidden layer

FSB

Deep L-layer Neural Network

- A little bit more complex model than previous one is a tree layer neural network (it is a neural network with two hidden layers):

Two hidden layers Neural Network



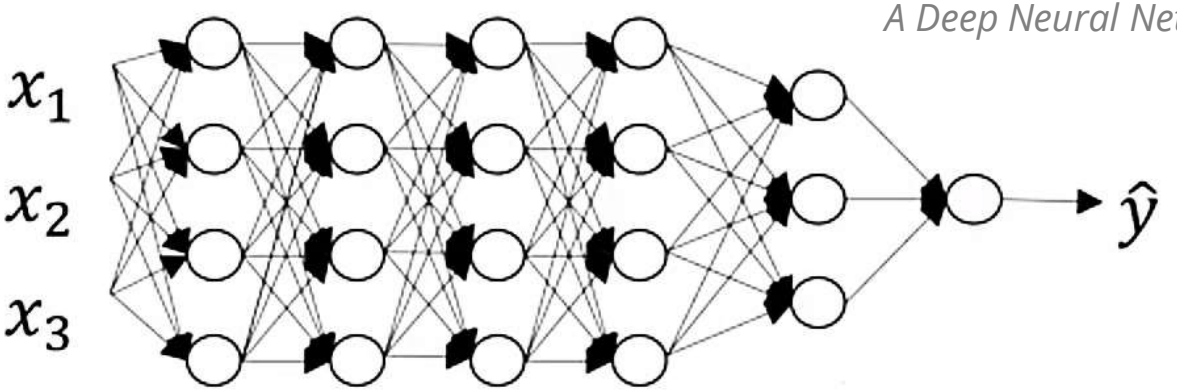
2 hidden layers

FSB

Deep L-layer Neural Network

- Even more complex neural network, which we can call **deep** neural network, is for example, a six layer neural network (or neural network with five hidden layers):

A Deep Neural Network

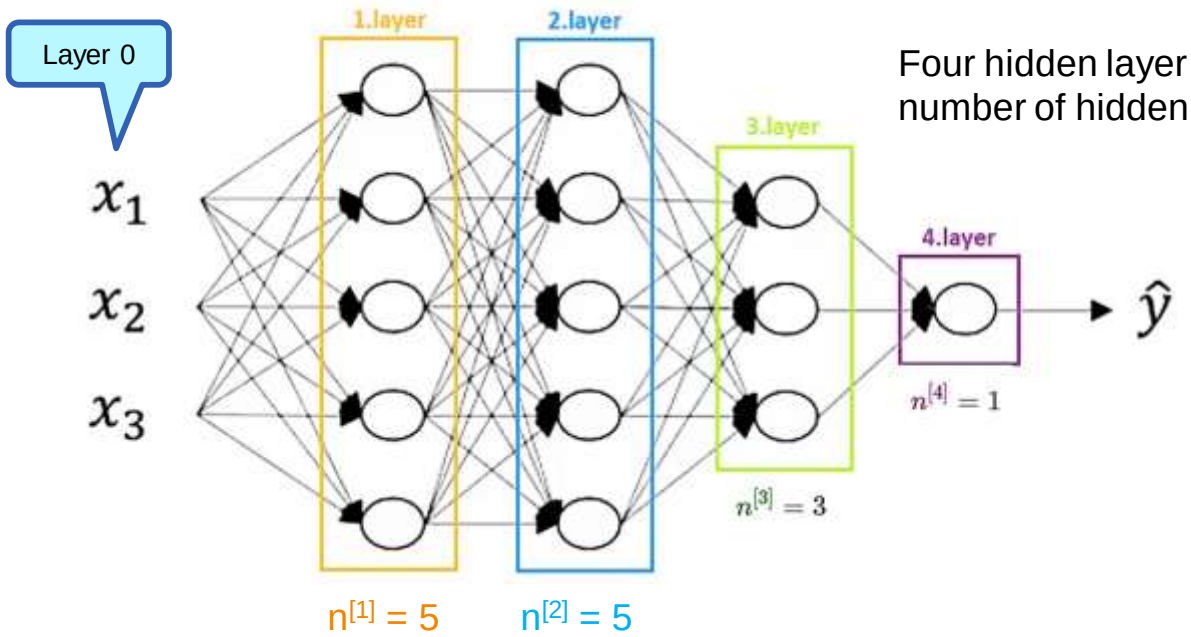


5 hidden layers

FSB

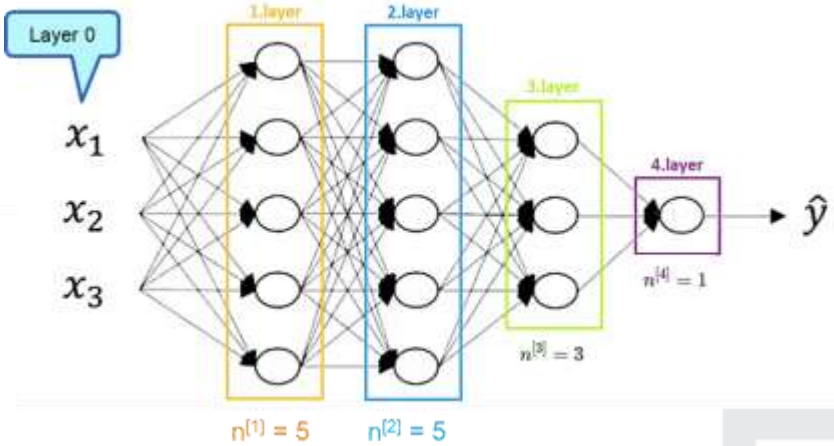
How do we count layers in a neural network?

- When counting layers in a neural network we count hidden layers as well as the output layer, but **we don't count an input layer**.



Four hidden layer Neural Network with a number of hidden units in each layer

How do we count layers in a neural network?



- L to denote the number of layers in a neural network
  - in this neural network $L = 4$

- $n^{[l]}$ to denote a number of layers in the lth layer
  - $n^{[1]} = 4$, there are four units in the first layer
  - $n^{[2]} = 4$, there are four units in the second layer
  - $n^{[3]} = 3$, there are three units in the thirs layer
  - $n^{[4]} = 1$, this neural network outputs a scalar value
  - $n^{[0]} = n_x = 3$ because input vector, feature vector, has three features

- $a^{[l]} = g(z^{[l]})$ to denote activation functions in the lth layer
  - $x = a^{[0]}$

- $\mathbf{W}^{[l]}$ to denote weights for computing $z^{[l]}$

Forward propagation in a deep network

- Given **X** input of (*nxm*) size and DNN of 4 layers, forward propagation computed:

Non – Vectorized

$$z^{[l]} = \mathbf{W}^{[l]} a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g^{[l]}(z^{[l]})$$

Dimensions are:

$$(n^{[1]}, 1) = (n^{[l]}, n^{[l-1]})(n^{[l-1]}, 1) + (n^{[l]}, 1)$$

$$(n^{[l]}, 1) = (n^{[l]}, 1)$$

Vectorized

$$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]} \mathbf{A}^{[l-1]} + b^{[l]}$$

$$\mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$$

Dimensions of these matrices are:

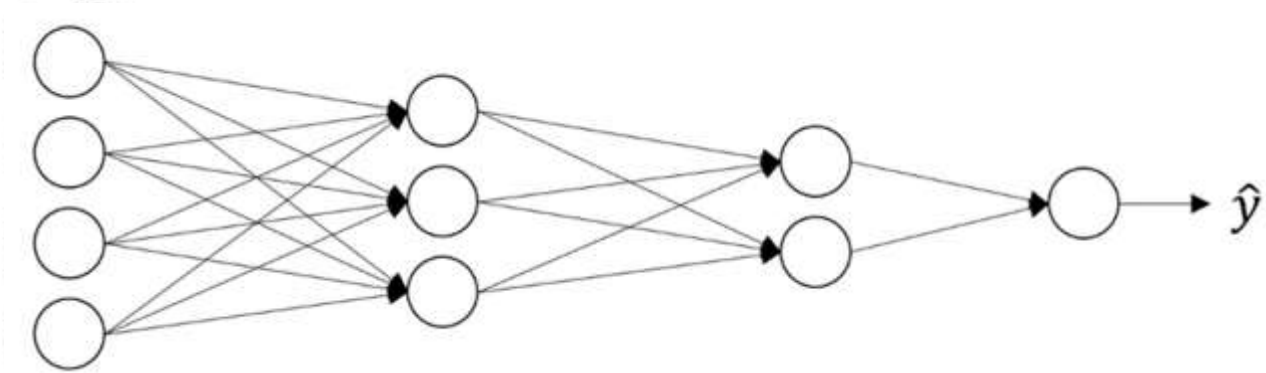
$$(n^{[l]}, m) = (n^{[l]}, n^{[l-1]})(n^{[l-1]}, m) + (n^{[l]}, 1)$$

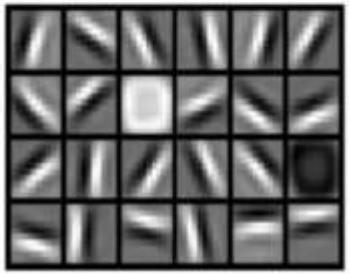
$$(n^{[l]}, m) = (n^{[l]}, m)$$

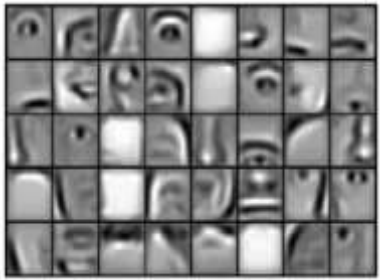
Why deep representation?

- We've heard that neural networks work really well for a lot of problems. However, neural networks doesn't need only to be big. Neural Networks also need to be deep or to have a lot hidden layers.
- If we are, for example, building a system for an image classification, here is what a deep neural network could be computing. The input of a neural network is a picture of a face. The first layer of the neural network could be a feature detector, or an edge detector. So, the first layer can look at the pictures and find out where are the edges in the picture. Then, in next layer those detected edges could be grouped together to form parts of faces. By putting a lot of edges it can start to detect different parts of faces. For example, we might have a low neurons trying to see if it's finding an eye or a different neuron trying to find part of a nose. Finally, putting together eyes, nose etc. it can be recognised different faces.

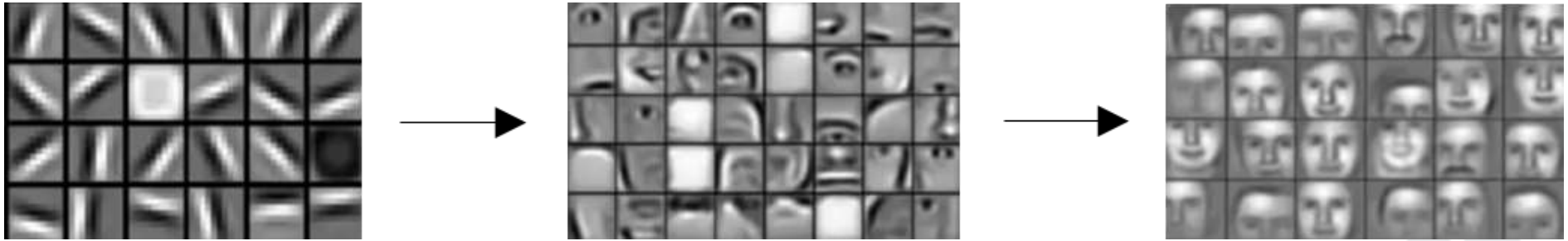
Why deep representation?







Why deep representation?

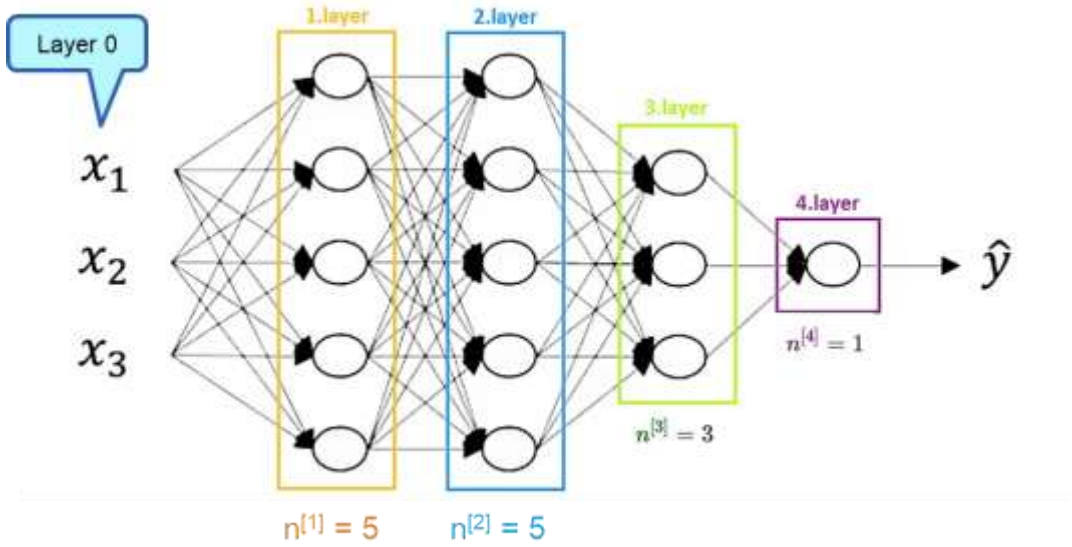


Example of feature detection. Simpler features are detected in the first layers and more complex features are detected in the later layers

Earlier layers learn lower level simple features and then later deep layers put together the simpler things it has detected in order to detect more complex things, so that a deep neural network can do some really complex things.

FSB

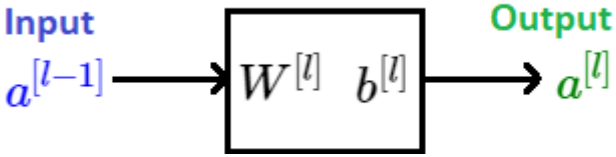
Building a Deep Neural Network



- Layer *l*: $w^{[l]}$, $b^{[l]}$
- Forward: Input $a^{[l-1]}$, output $a^{[l]}$

Building a Deep Neural Network

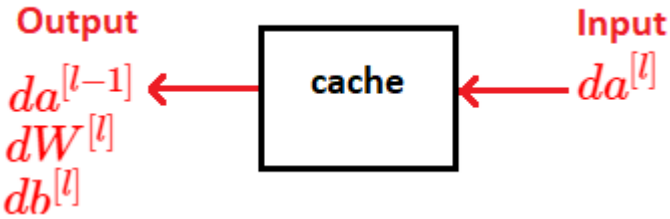
- Diagram of a Forward pass through layer *l*



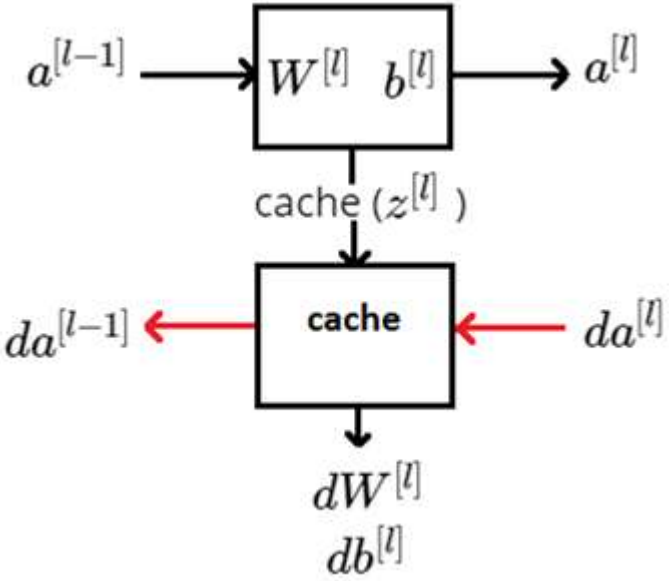
$$z^{[l]} = \mathbf{W}^{[l]} a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g(z^{[l]})$$

- Diagram of a Backward pass through layer *l*

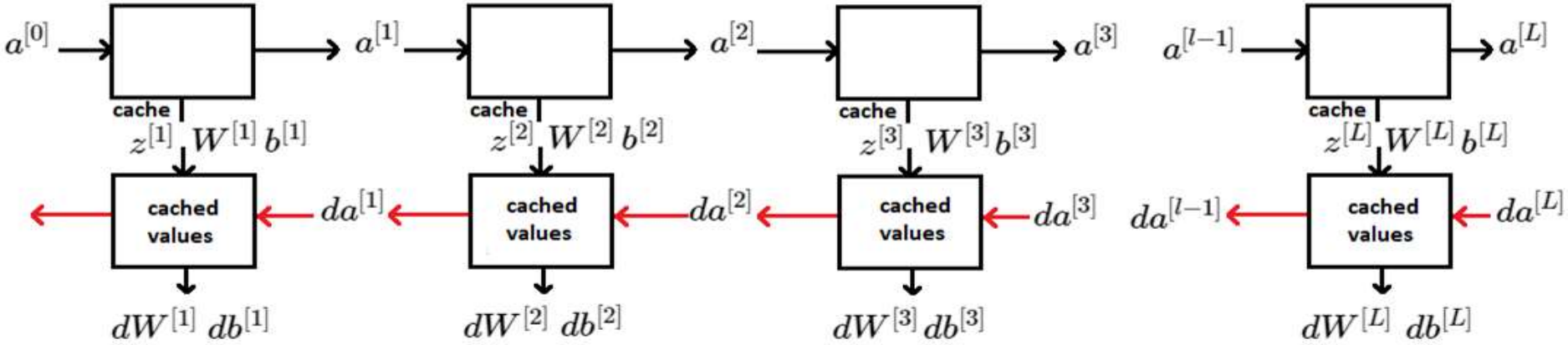


- Diagram of a Forward and Backward pass through layer *l*



Building a Deep Neural Network

- Diagram for a Forward and a Backward pass paths for L layer Neural Network



Update parameters:

$$\mathbf{W}^{[l]} := \mathbf{W}^{[l]} - \alpha \mathbf{dW}^{[l]}$$

$$b^{[l]} := b^{[l]} - \alpha db^{[l]}$$

Vectorized

$$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]}\mathbf{A}^{[l-1]} + b^{[l]}$$

$$\mathbf{A}^{[l]} = g(\mathbf{Z}^{[l]})$$

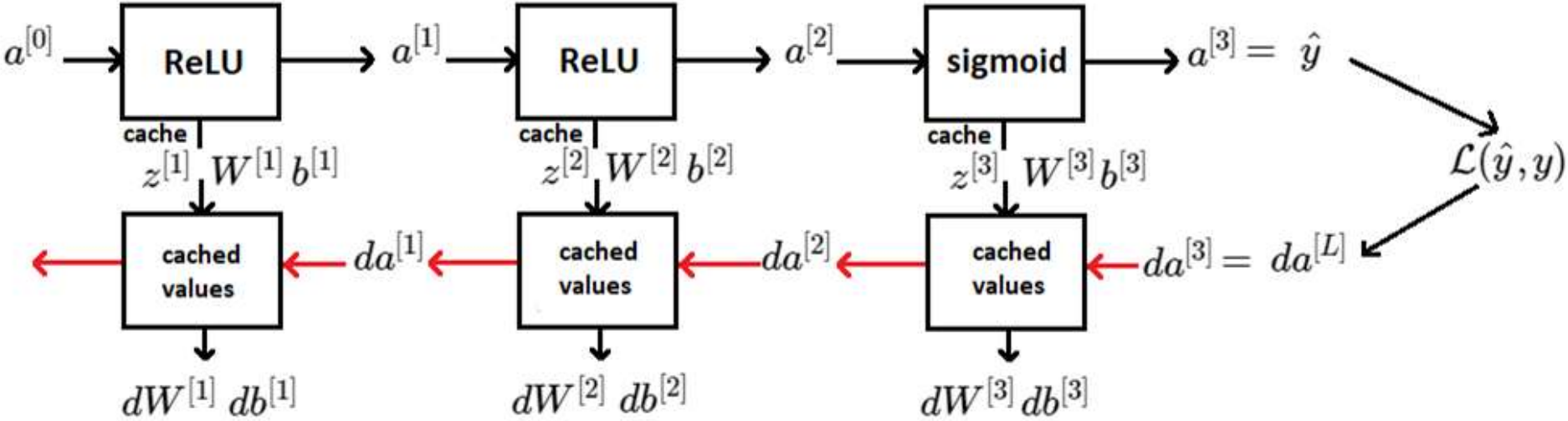
$$\mathbf{dZ}^{[l]} = \mathbf{dA}^{[l]} * g'^{[l]}(\mathbf{Z}^{[l]})$$

$$\mathbf{dW}^{[l]} = \mathbf{dZ}^{[l]}\mathbf{A}^{[l-1]T}$$

$$db^{[l]} = \frac{1}{m} np.sum(\mathbf{dZ}^{[l]}, axis = 1, keepdims = True)$$

$$\mathbf{dA}^{[l-1]} = \mathbf{W}^{[l]T}\mathbf{dZ}^{[l]}$$

Example: A 2-layer neural network



matrix $\mathbf{X}$ where we stack all training examples horizontally. $x = a^{[0]}$

$$da^{[L]} = -\frac{y}{a} + \frac{1-y}{1-a}$$

$$\mathbf{dA}^{[L]} = \left[-\frac{y^{(1)}}{a^{(1)}} + \frac{1-y^{(1)}}{1-a^{(1)}} \quad -\frac{y^{(2)}}{a^{(2)}} + \frac{1-y^{(2)}}{1-a^{(2)}} \quad \cdots \quad -\frac{y^{(m)}}{a^{(m)}} + \frac{1-y^{(m)}}{1-a^{(m)}} \right]$$

Parameters and Hyperparameters of a DNN

- a learning rate – α
- a number of iteration
- a number of layers L
- a number of hidden units $n^{[1]}, n^{[2]}, \ldots n^{[L]}$
- a choice of activation function

Model parameters are estimated from data during the training step and model hyperparameters are manually set earlier and are used to assist estimation model parameter.

These are parameters that control our parameters $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, W^{[3]}, b^{[3]} \ldots$ and we call them hyperparameters. In deep learning there are also these parameters: momentum, batch size, number of epochs etc…

We can see that **hyperparameters** are the variables that determines the network structure and the variables which determine how the network will be trained. Notice also that **hyperparameters** are set before training or before optimizing weights and biases.

Building a Deep Neural Network from scratch in Python

https://datahacker.rs/deep-neural-network-from-scratch-python/

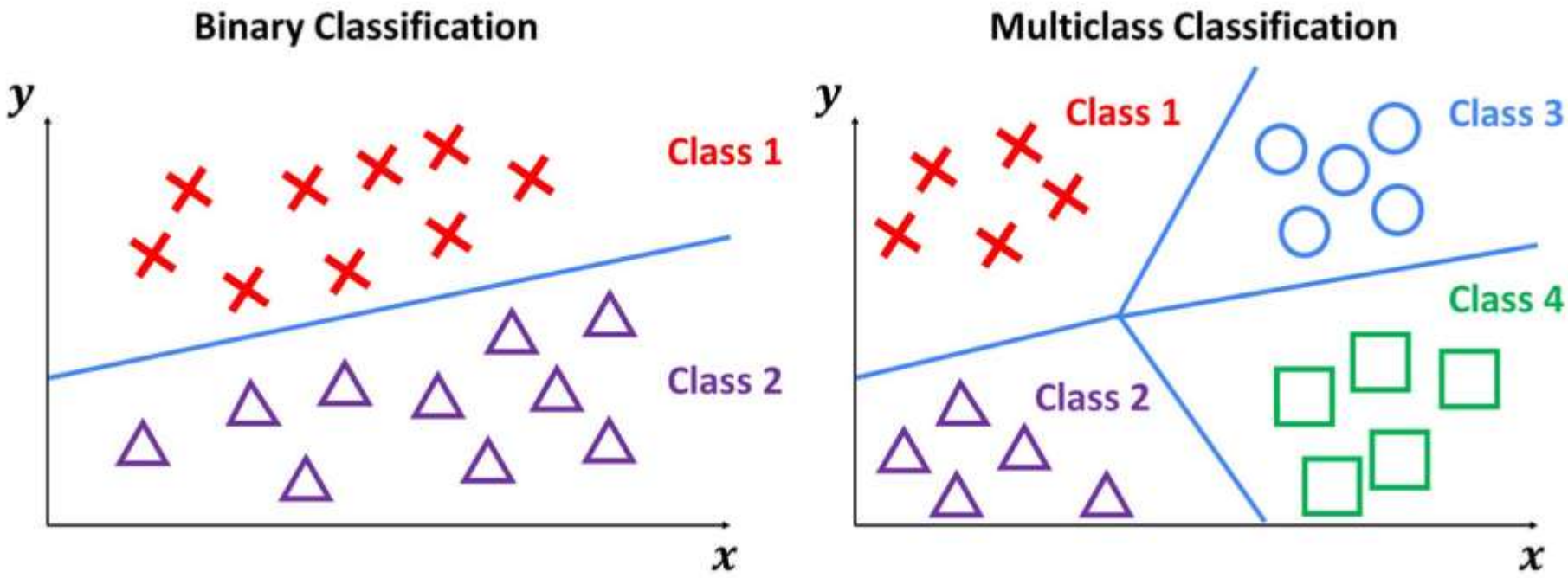
FSB

Multiclass Classification & Softmax Function

FSB

Multiclass classification

- A multiclass classification is a classification problem where y can take only a small number of **discrete categories**.



The softmax function

- **Recall** that logistic regression produces a decimal between 0 and 1. For example, a logistic regression output of 0.8 from a dog/cat classifier suggests an 80% chance of an image of a dog and a 20% chance of an image of a cat.

$$z = \vec{\mathbf{w}} \cdot \vec{\mathbf{x}} + b$$

$$a = g(z) = \frac{1}{1 + e^{-z}} = P(y = 1 \mid \vec{\mathbf{x}})$$

- The **softmax function** extends this idea by assigning decimal probabilities to each class in a multi-class problem. It is a generalization of logistic regression, which is a binary classification algorithm to the multiclass classification contexts.

FSB

Softmax regression

- Given an example of ***y*** takes four possible outputs 1, 2, 3, or 4. What softmax regression will do?
- The parameters of softmax regression z_1, z_2, z_3 and z_4:

$$z_1 = \vec{\mathbf{w}}_1 \cdot \vec{\mathbf{x}} + b_1 \qquad a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = 1 \mid \vec{\mathbf{x}})$$

$$z_2 = \vec{\mathbf{w}}_2 \cdot \vec{\mathbf{x}} + b_2 \qquad a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = 2 \mid \vec{\mathbf{x}})$$

$$z_3 = \vec{\mathbf{w}}_3 \cdot \vec{\mathbf{x}} + b_3 \qquad a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = 3 \mid \vec{\mathbf{x}})$$

$$z_4 = \vec{\mathbf{w}}_4 \cdot \vec{\mathbf{x}} + b_4 \qquad a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = 4 \mid \vec{\mathbf{x}})$$

Softmax regression

- ***y*** can take *N* possible values, so y can be 1, 2, 3, and so on up to *N*.
- *k* to index the summation because *j* refers to a specific fixed number like *j*=1. The value of a_j is interpreted as the model's estimate that ***y*** given the input features ***x***.

$$a_1 + a_2 + \ldots + a_N = 1$$

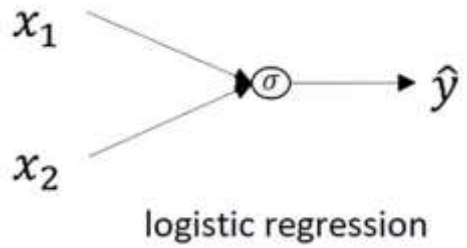
$$z_j = \vec{\mathbf{w}}_j \cdot \vec{\mathbf{x}} + b_j$$

$$j = 1, \ldots, N$$

$$a_j = \frac{e^{z_j}}{\sum_{k=1}^{N} e^{z_k}} = \mathrm{P}(\mathrm{y} = j \mid \vec{\mathrm{x}})$$

The cost function for Logistic regression

- Recall from Logistic Regression with binary classification (2 classes):



$$a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1 \mid \vec{\mathrm{x}})$$

$$a_2 = 1 - a_1 \quad = P(y=0 \mid \vec{\mathrm{x}})$$

- Loss of logistic regression:

$$\boxed{loss = -y \log a_1 - (1-y)\log(1-a_1)}$$

Note that here this term $(1 - a_1)$ is also equal to a_2. Therefore we can rewrite or simplify the loss for logistic regression in the following way. We can say that if y is equal to 1, the loss is $-y \log a_1$. On the other hand if y is equal to 0, then the loss is $-y \log a_2$. Then, the cost function for all the parameters in the model is the average loss, average over the entire training set.

$$\boxed{J(\vec{\mathbf{w}}, b) = \text{ average loss}}$$

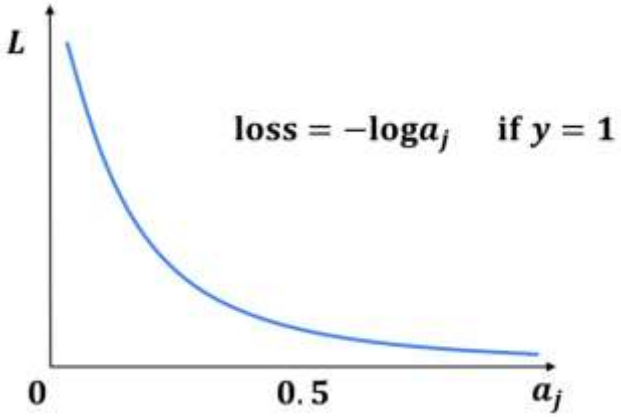
The cost function for Softmax regression

- Softmax regression = predicted SR output:

$$a_N = \frac{e^{z_N}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = P(y = N \mid \vec{\mathbf{x}})$$

- Loss of softmax regression:

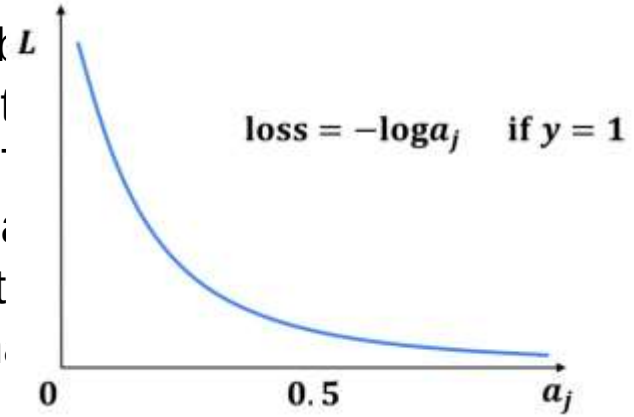
$$loss\,(a_1, \ldots, a_N, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ -\log a_2 & \text{if } y = 2 \\ & \vdots \\ -\log a_N & \text{if } y = N \end{cases}$$



FSB

The cost function for Softmax regression

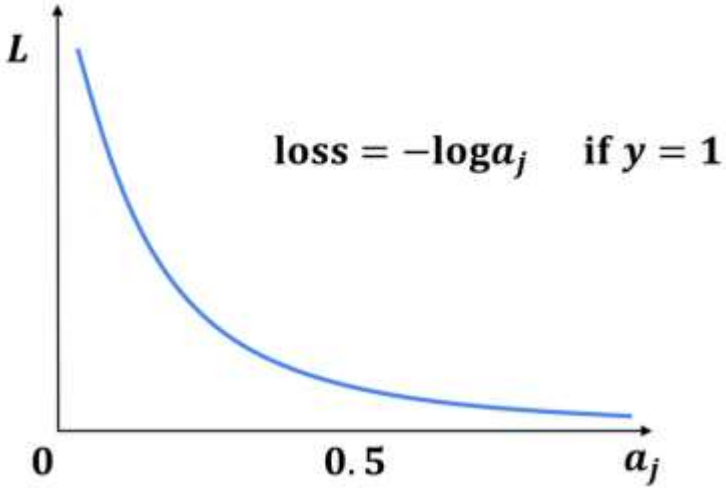
- We can see that If a_j was very close to 1, then it l
  the curve and the loss will be very small. But if it f
  50% chance then the loss gets a little bit bigger. T
  the loss. This incentivizes the algorithm to make a
  close to 1 as possible. Because whatever the act
  the algorithm to say that the chance of y being th



- So to summarize, in the case of multiclass classification, what we want to see is a prediction of which class the network “thinks” the input represents. This distribution returned by the softmax activation function represents confidence scores for each class and will always add up to 1.

The cost function for Softmax regression

- We can see that If a_j was very close to 1, then it belongs to the right part of the curve and the loss will be very small.
- But if it thought, say, a_j had only a 50% chance then the loss gets a little bit bigger.
- The smaller a_j is, the bigger the loss.
- This incentivizes the algorithm to make a_j as large as possible, as close to 1 as possible. Because whatever the actual value of y was, you want the algorithm to say that the chance of y being that value was pretty large.



In the case of multiclass classification, what we want to see is a prediction of which class the network “thinks” the input represents. This distribution returned by the softmax activation function represents confidence scores for each class and will always add up to 1.

Softmax function in Python

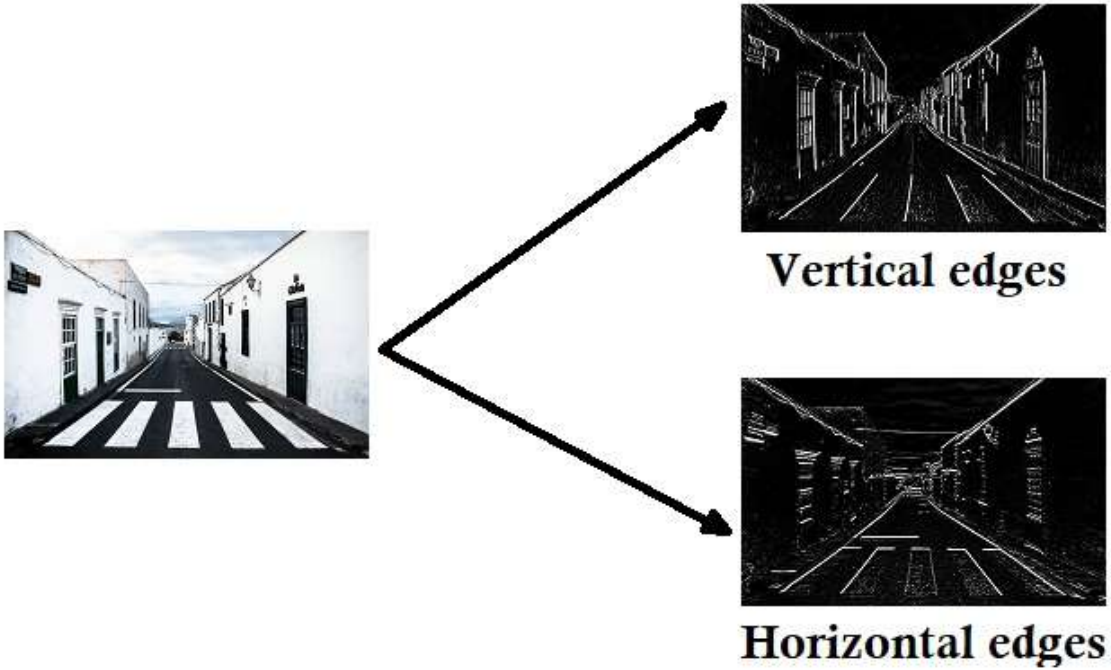
- https://colab.research.google.com/drive/1fjF6RZcqCVjkyRX7s0XZTbChwseFezmH?usp=sharing

Convolutional Neural Networks

FSB

Convolutional operation

- Edge detection – vertical and horizontal edge detection



FSB

How can we discover the edges in the picture?

- Given a gray image 6x6x1, **Detecting a vertical edge using a filter (kernel) constructed with a 3x3 matrix. The convolution operation is marked with an asterisk $*$.**

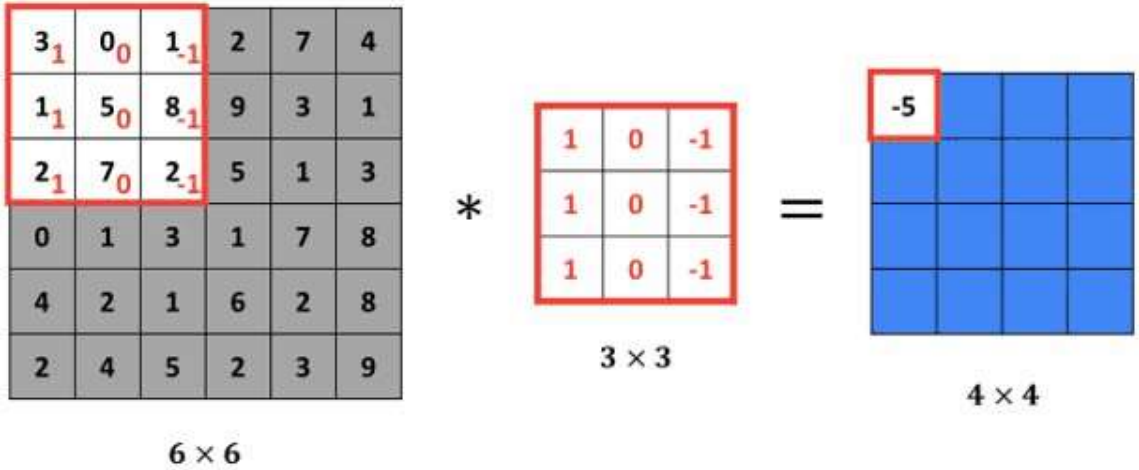
3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

1	0	-1
1	0	-1
1	0	-1

3×3

FSB

Convolutional Process



$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

FSB

Filters

1	0	-1
1	0	-1
1	0	-1

Vertical

1	1	1
0	0	0
-1	-1	-1

Horizontal

1	0	-1
2	0	-2
1	0	-1

Sobel filter

3	0	-3
10	0	-10
3	0	-3

Scharr filter

FSB

Example

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

6 x 6

*

1	0	-1
1	0	-1
1	0	-1

3 x 3

=

-0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

4 x 4

$$10 \times 1 + 10 \times 0 + 10 \times (-1) + 10 \times 1 + 10 \times 0 + 10 \times (-1) + 0 \times 1 + 0 \times 0 + 0 \times (-1) = 0$$

Edge detection – an original image (left), a filter (in the middle), a result of a convolution (right)

TensorFlow library

- If we want to implement this process **in a certain programming language, we will use "of-the shelf" functions instead of a star that denotes a convolution.** For example, there is a *tf.nn.conv2d* function in the TensorFlow library.

FSB

CNN Padding

- If we convolve an n×n image with an f×f filter, what are the dimensions of the output matrix? If we have an n×n, and convolve that with a f×f, then the dimension of the output will be : $(n-f+1)\times(n-f+1)$.

0	0	0	0	0	0	0	0
0	3	3	4	4	7	0	0
0	9	7	6	5	8	2	0
0	6	5	5	6	9	2	0
0	7	1	3	2	7	8	0
0	0	3	7	1	8	3	0
0	4	0	4	3	2	2	0
0	0	0	0	0	0	0	0

$6 \times 6 \rightarrow 8 \times 8$

$*$

1	0	-1
1	0	-1
1	0	-1

3×3

$=$

-10	-13	1			
-9	3	0			

6×6

Valid and same convolutions

- Valid convolutions ⇔ no padding

size of an input image	size of a filter	padding	size of an output image
$n \times n$	$f \times f$	no padding, p=0	$n - f + 1 \times n - f + 1$
6	3		$6 - 3 + 1 = 4 \Rightarrow 4 \times 4$

- Same convolution ⇔ pad so that output size is the same as the input size

size of an input image	size of a filter	padding	size of an output image
$n \times n$	$f \times f$	$p = 1$	$n + 2p - f + 1 \times n + 2p - f + 1$
6	3		$6 + 2 - 3 + 1 = 6 \Rightarrow 6 \times 6$

FSB

CNN Strided Convolution

- *Convolutions with a stride of two*

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3^3	4^4	6^4
3	2	4	1	9^1	8^0	3^2
0	1	3	9	3^{-2}	1^0	4^3

X

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

=

91	110	83
69	91	127
44	72	74

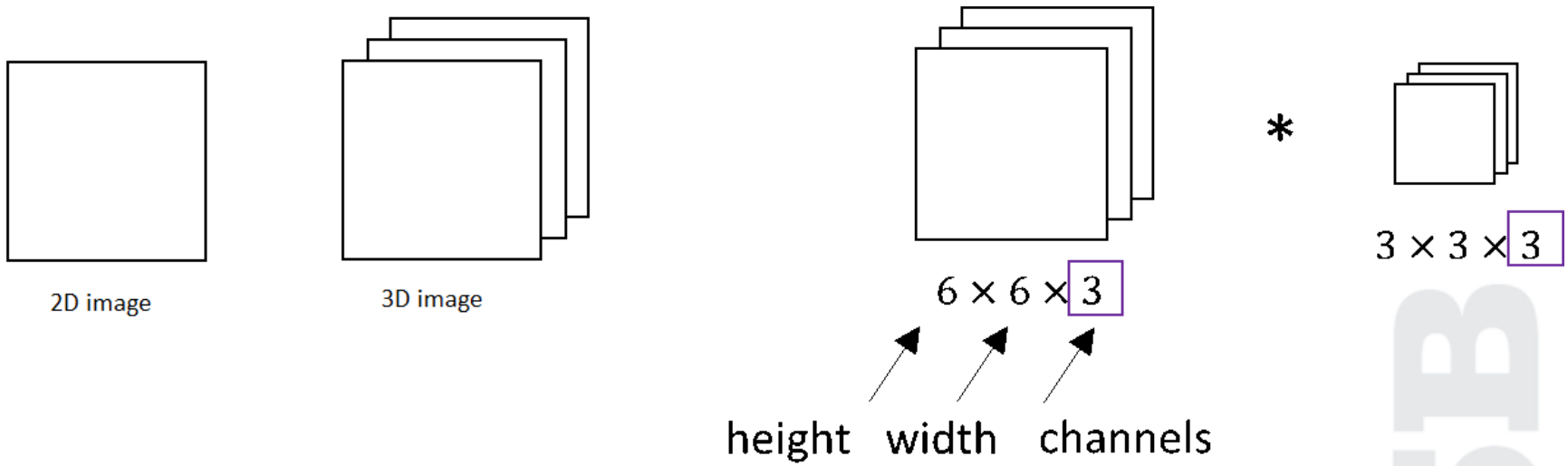
3×3

Strided convolution

- ***Correlation*** *is a measurement of the similarity between two signals/sequences.*
- ***Convolution*** *is a measurement of the effect of one signal on the other signal.*

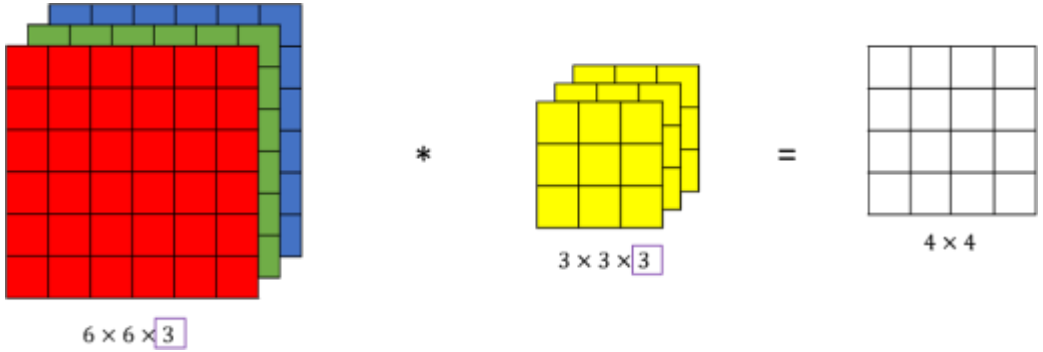
size of an input image	size of a filter	padding	stride	size of an output
$n \times n$	$f \times f$	p	s	$\lfloor \frac{n-f+2p}{s} \rfloor + 1 \times \lfloor \frac{n-f+2p}{s} \rfloor + 1$

CNN Convolution On RGB Images



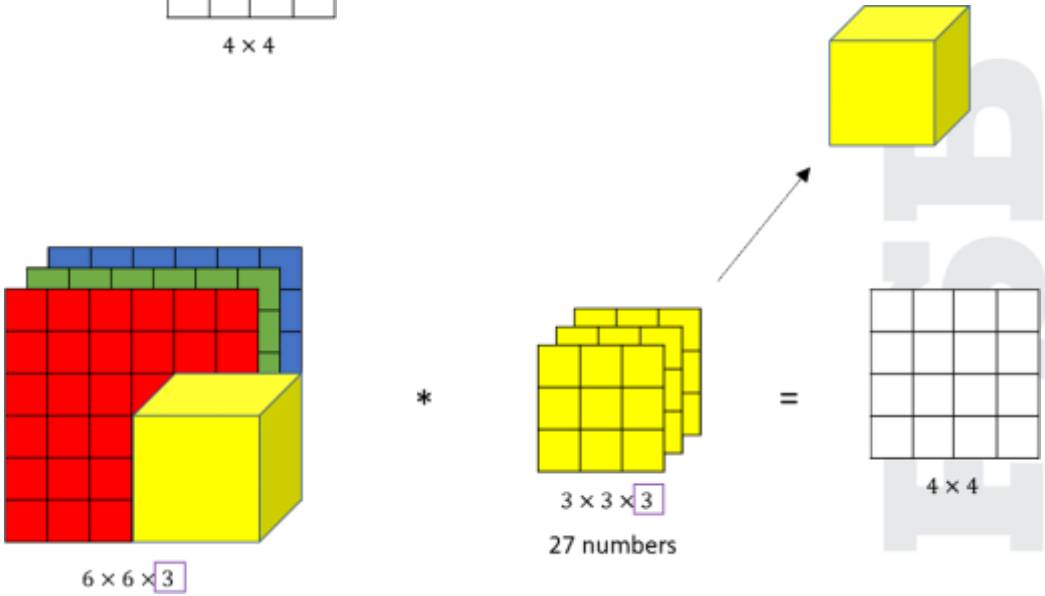
- In order to detect edges or some other feature in this image, we convolve it not with a 3×3 filter, but now with a 3 – dimensional filter. That's gonna be a 3×3×3, so the filter itself will also have three layers corresponding to red, green and blue channels.

Convolutions on RGB image

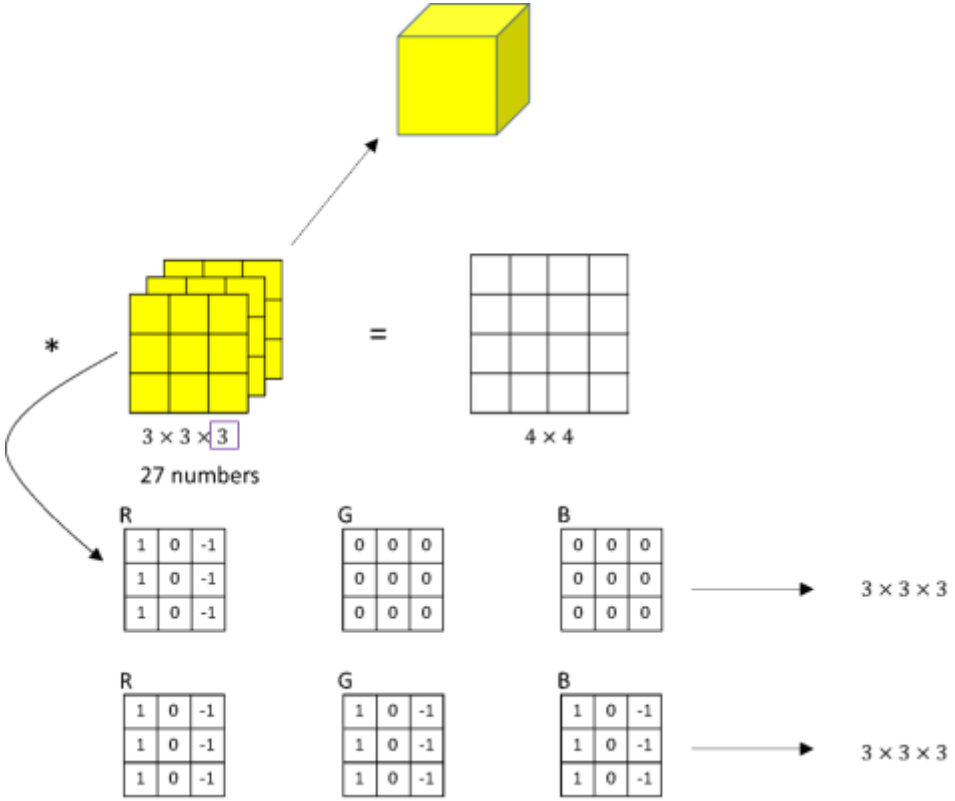


- *RGB image, corresponding filter for convolution and the result of a convolution*

- When we apply 3×3×3 filter on the RGB image it is as we implement the volume



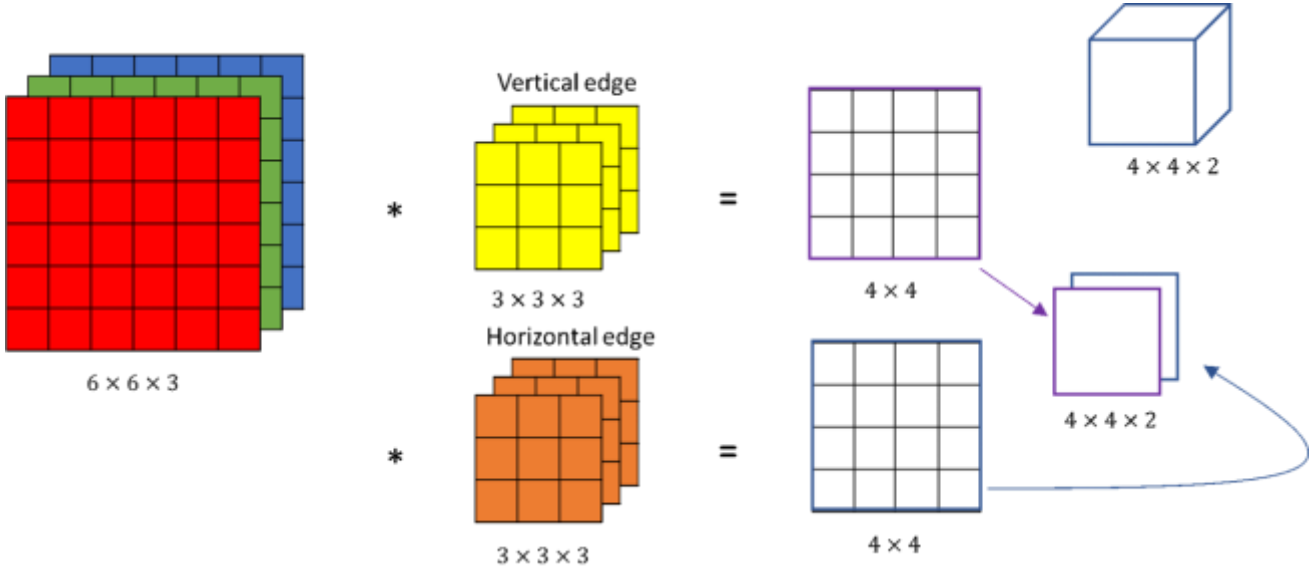
Why our filter has tree channels and what are the coefficients in that filter ?



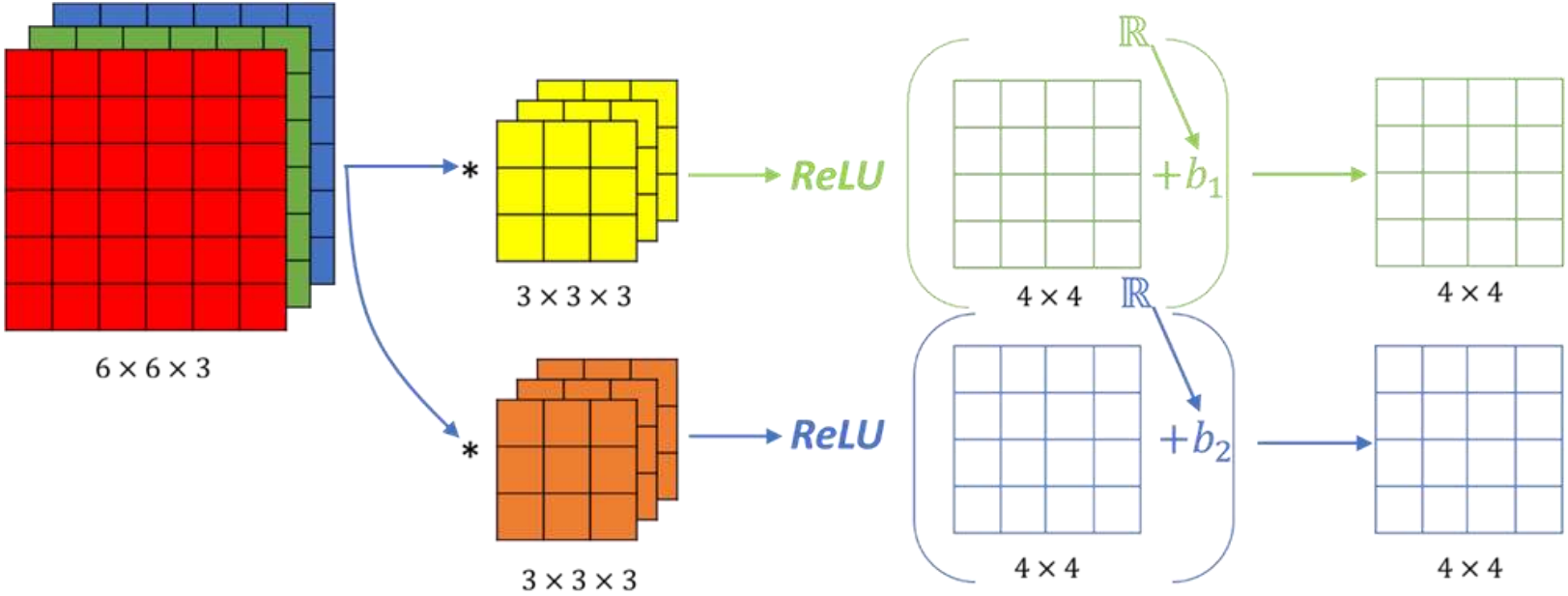
FSB

When we convolve with two different filters simultaneously

- The idea of convolution on volumes turns out to be really powerful. Only a small part of it is that you can now operate directly on RGB images with 3 channels, but even more important is that you can now detect 2 features like horizontal and vertical edges. Furthermore, there can be 10 or maybe 128 or maybe several hundred different features. Finally the output will then have a number of channels equal to the number of features we are trying to detect.

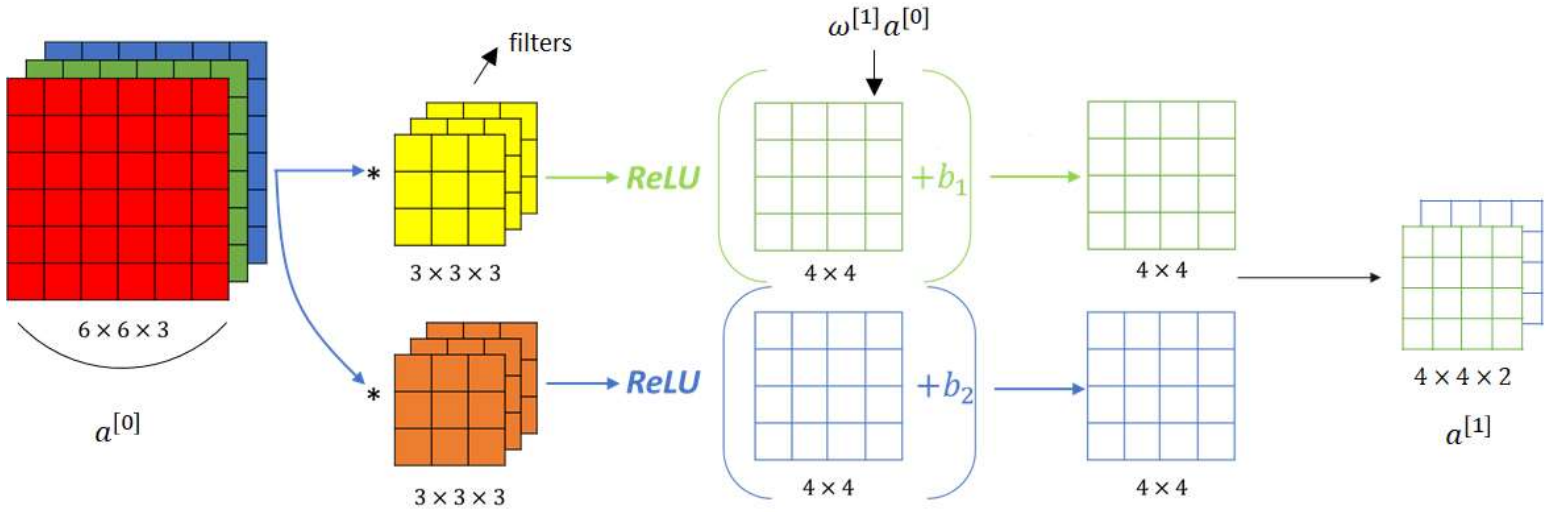


CNN One Layer of A ConvNet



FSB

CNN One Layer of A ConvNet

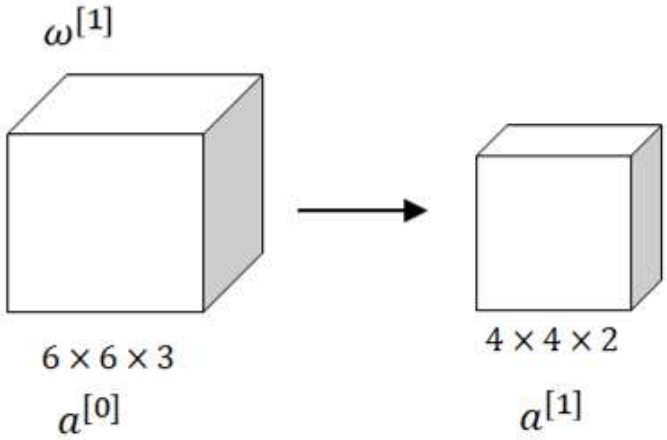


A Convolutional layer

- Using the analogy from neural networks, our input here is this 6×6×3 volume and these convolutional filters are like W[1].
- The convolution is really: apply the linear operation add the biases and apply ReLU.

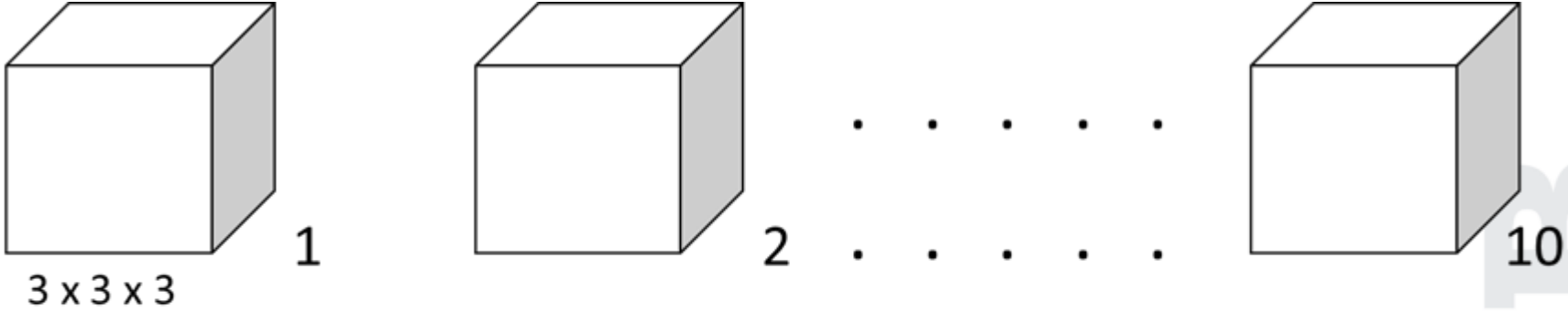
$$Z^{[1]} = \omega^{[1]} a^{[0]} + b^{[1]}$$

$$a^{[1]} = g^{(Z^{[1]})}$$



FSB

Number of parameters in one layer



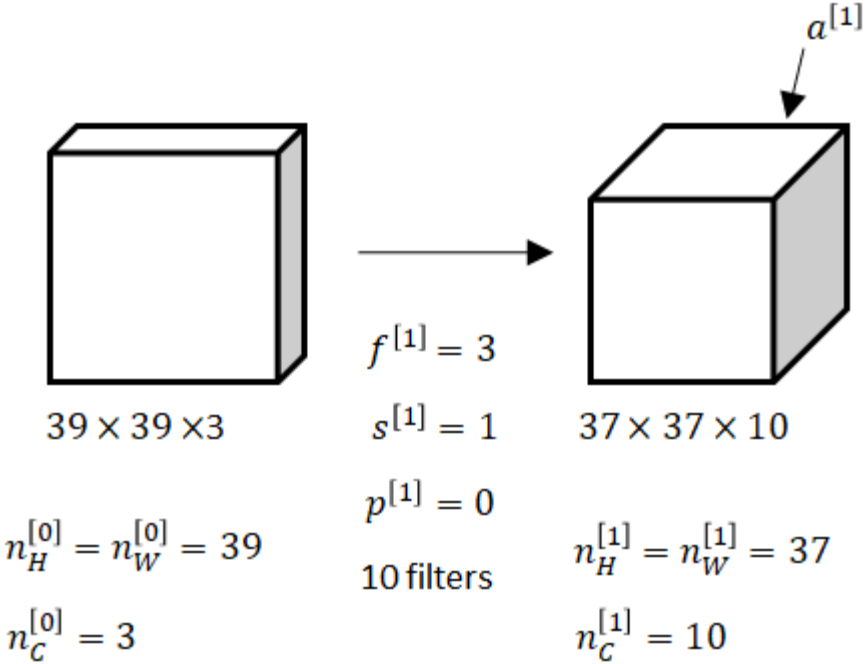
27 parameters
+ bias
= 28 parameters

280 parameters

FSB

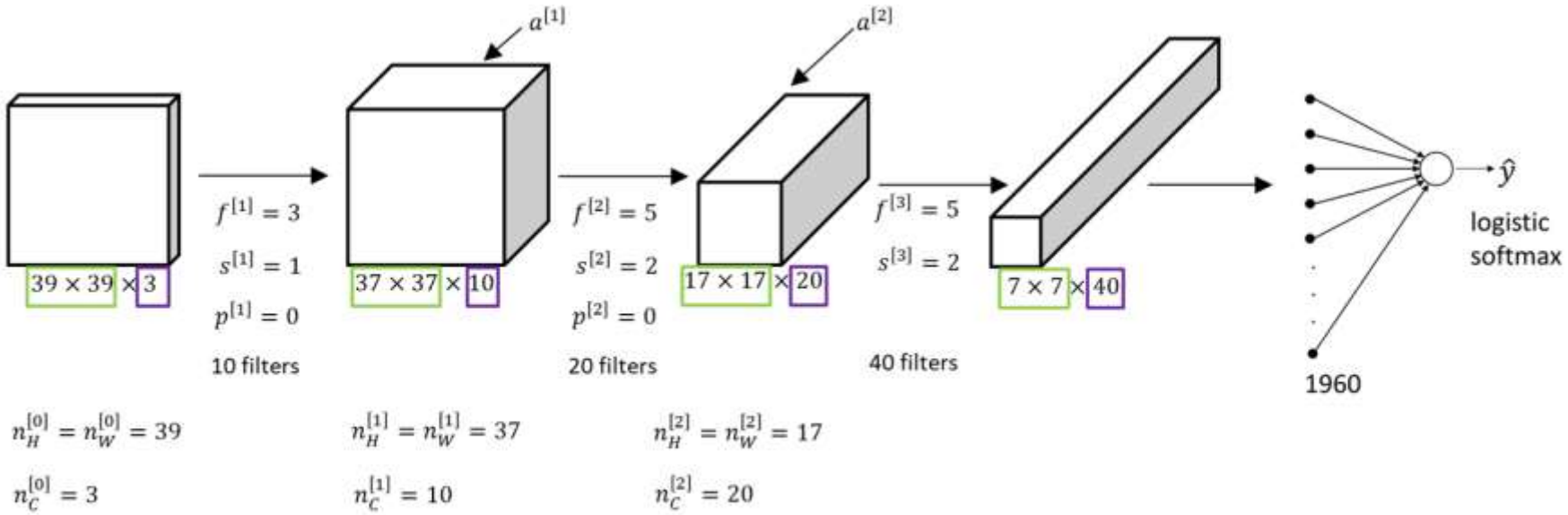
CNN An Example of A Convolutional Neural Network

- Build an example of a ConvNet



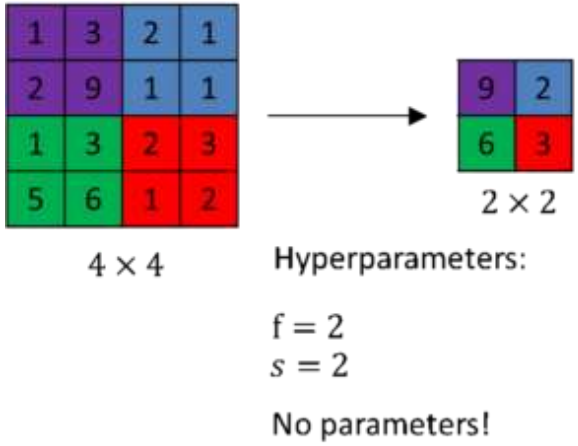
An example of a convolution

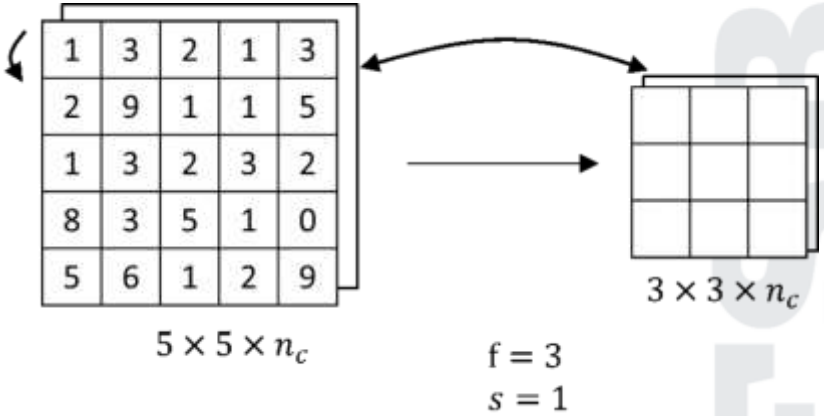
- **Hyperparameters**: deciding what's the filter size, what's the stride, what's the padding and how many filters to use.



CNN Pooling Layers

- **Maxpooling** with a 2×2 filter and a stride of 2.
- Two main reasons that peope use Maxpooling:
  - It's been found in a lot of experiments to work well.
  - It has no parameters to learn. There's actually nothing for the gradient descent to learn. Once we've fixed f and s, it's just a fixed computation and gradient descent doesn't change anything.

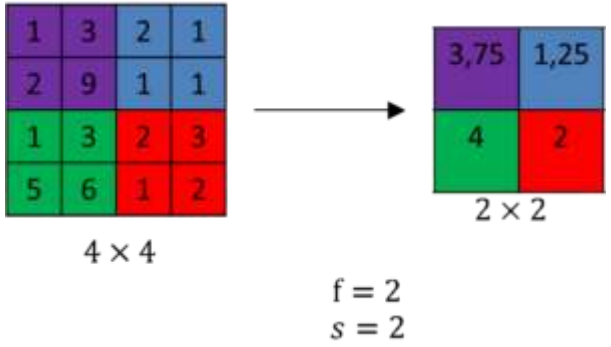


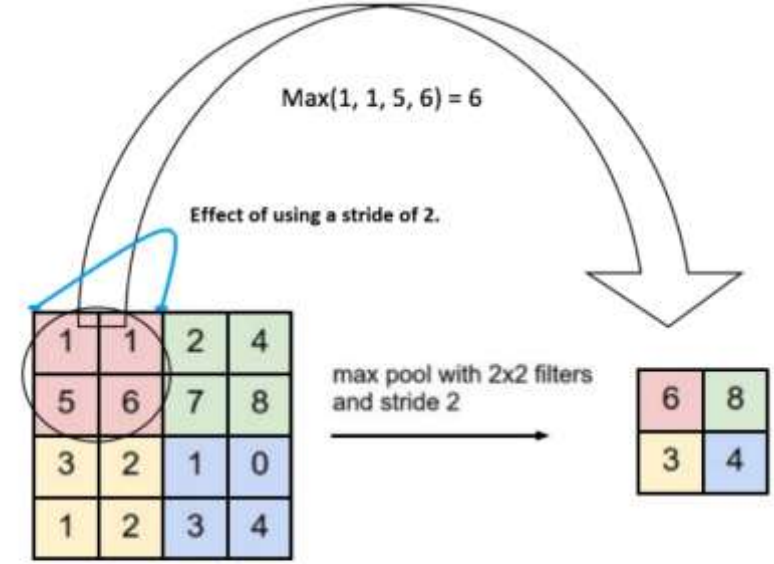


Result of the Maxpooling applied on a 3D volume

CNN Pooling Layers

- **Average pooling** with a 2×2 filter and a stride of 2.





We do the pooling with 2x2 filters, so we will devide an input image on 2x2 regions, and we will use a stride of 2.
Because we are using a stride of 2, these regions don't overlap.

CNN Pooling Layers

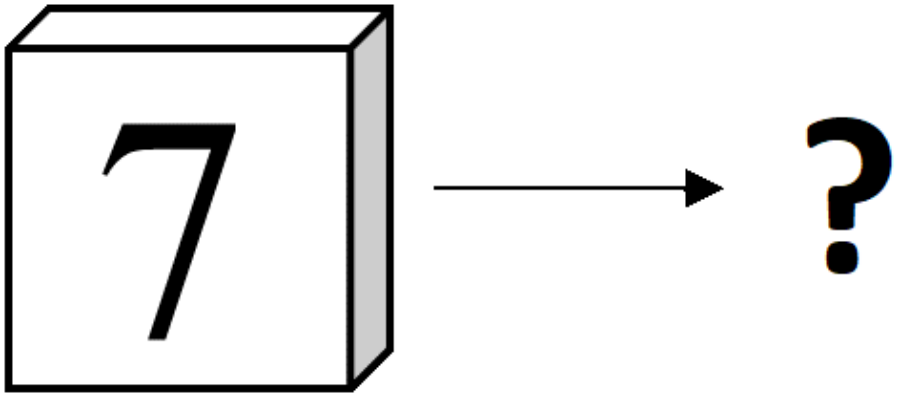
- *Hyperparmeters for pooling*

filter size	stride	pooling	type of pooling
$f \times f$	s	p	$Max\ pooling$
			or
		rarely used	$Averedge\ pooling$

FSB

Convolutional Neural Network – An Example

- *An example of an input to a CNN when doing a hand written digits recognition*

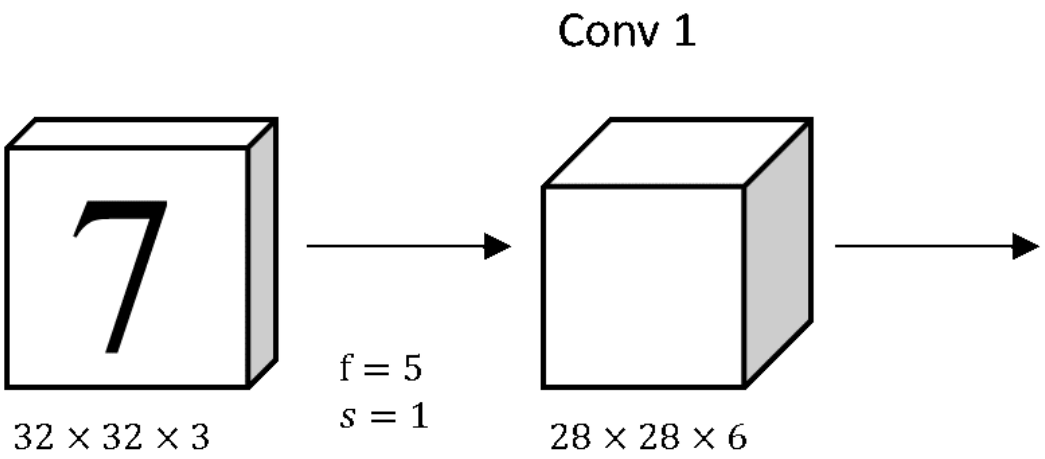


$32 \times 32 \times 3$

FSB

Convolutional Neural Network – An Example

- *An example of an input to a CNN when doing a hand written digits recognition*

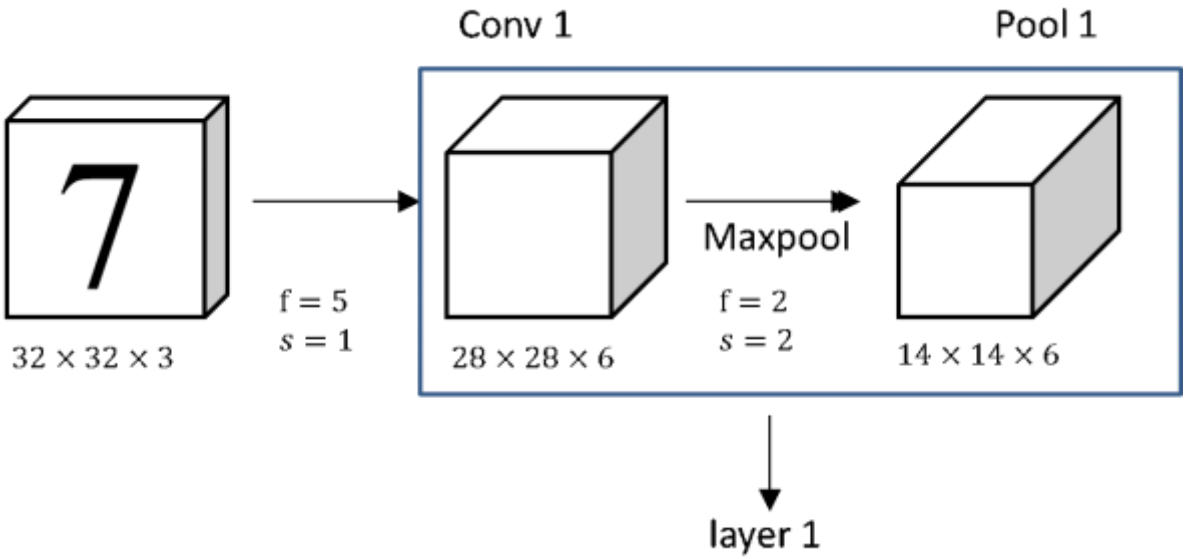


LeNet–5 neural network – the first Conv layer

FSB

Convolutional Neural Network – An Example

- *An example of an input to a CNN when doing a hand written digits recognition*

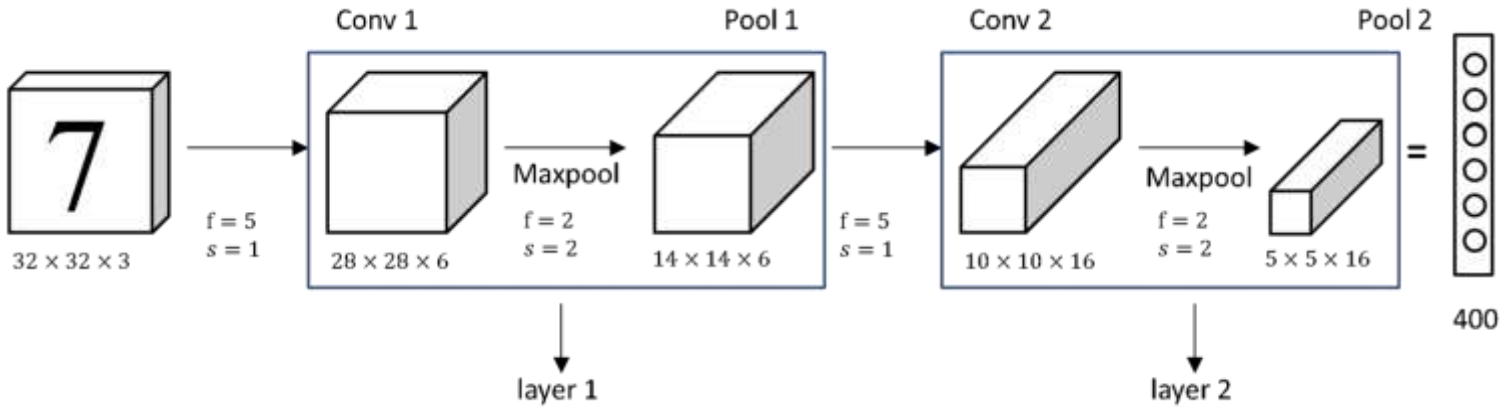


LeNet–5 – the first layer (the Conv and the Maxpooling layer)

FSB

Convolutional Neural Network – An Example

- *An example of an input to a CNN when doing a hand written digits recognition*

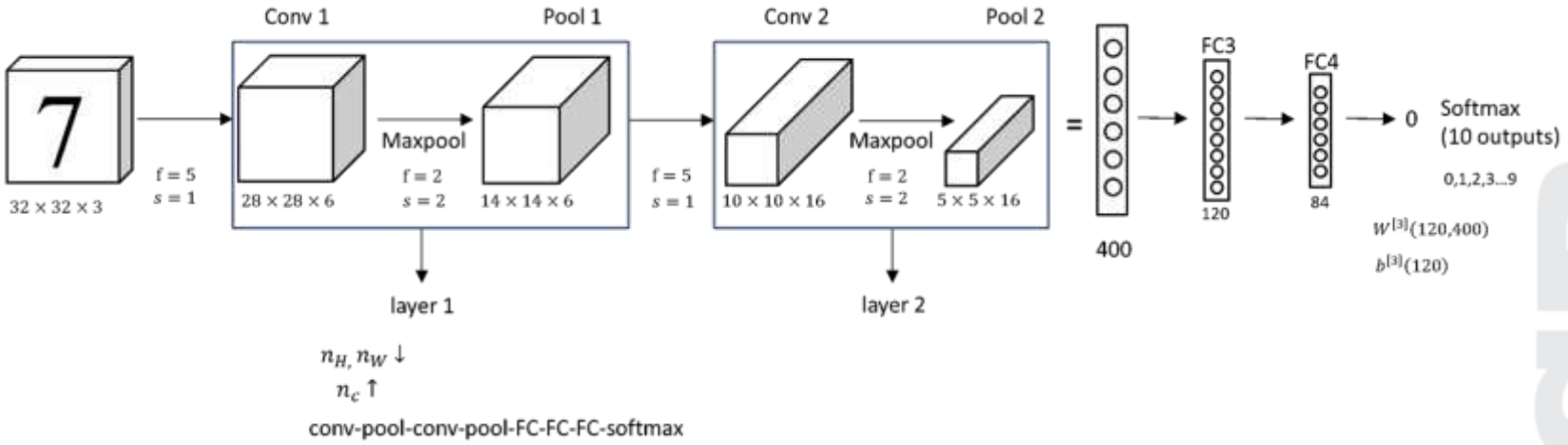


The first two layers of LeNet−5 neural network

FSB

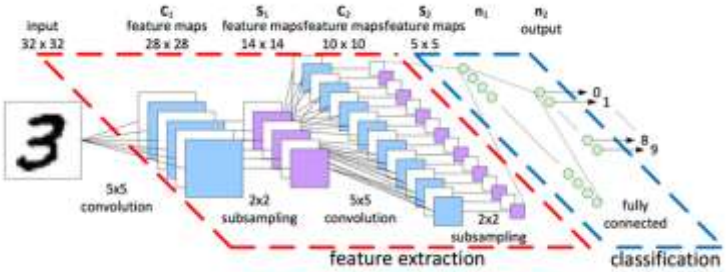
Convolutional Neural Network – An Example

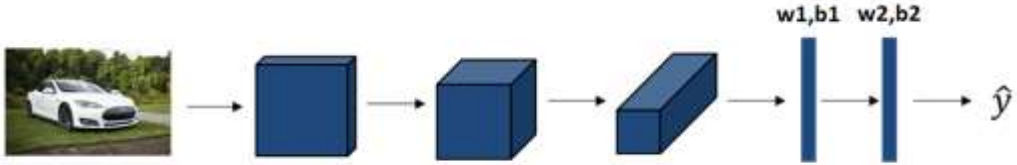
- *An example of an input to a CNN when doing a hand written digits recognition*



LeNet−5 neural network with Fullyconnected layers

Examples





We want to build a car detector and we have a training set as follows: x is an image and y can be binary labels for one of k classes. Let's say we've chosen a convolutional neural network structure that suits the image. That convolutional neural network consists of some convolutional and Poolinglayers. Then, we have Fullyconnected layers followed by a softmax output that gives us y^. The Convolutional layers and the Fully connected layers will have various parameters w, as well as biases b, so that lets us to define a cost function. We can compute the cost function J as the sum of losses of our neural networks predictions on our entire training set and it may be divided by m to get an average value. To train this neural network we can use a gradient descent or some other optimization algorithm, like gradient descent with momentum, in order to optimize all the parameters of the neural network and to reduce the cost function J. And if we do this, we can build a very effective car detector or some other detector.

Further Reading - References

https://datahacker.rs/deep-learning/

https://machinelearningmastery.com/when-to-use-mlp-cnn-and-rnn-neural-networks/

- CNN architectures
- RNNs
- Transformer
- Yolo models

FSB

Examples for MLP and CNN models

https://colab.research.google.com/drive/19at4IpNmLJnkK7G9eGpdcqFy-RlW-ill?usp=sharing

FSB